Khan Institute of

Computer Science and

Information Technology

Department of Computer Science

Practical Assignment No. 1 — Mini Project

Sentiment Analyzer Pro

Multi-Language AI-Powered Sentiment Analysis Application using ML.NET & Windows Forms

Submitted by: Muhammad Bilal

Roll No: 232201100

Submitted to: Sir Uzair Hassan

Department of Computer Science

March 30, 2026

0 Contents

1 Project Introduction **2**
- 1.1 Problem Statement . . . 2
- 1.2 Objective . . . 2

2 System Design **2**
- 2.1 Architecture Diagram . . . 3
- 2.2 Workflow Diagram . . . 3

3 Tools & Technologies **4**
- 3.1 Development Environment . . . 4
- 3.2 NuGet Packages Used . . . 4
- 3.3 External API . . . 4
- 3.4 Dataset . . . 5

4 Code Snippets **5**
- 4.1 Data Model Class . . . 5
- 4.2 ML.NET Training Pipeline . . . 6
- 4.3 Custom CSV Parser for IMDB Dataset . . . 6
- 4.4 Multi-Language Translation Service . . . 7
- 4.5 Sentiment Prediction . . . 8

5 Application Preview **8**
- 5.1 User Interface Overview . . . 8
- 5.2 UI Layout (Actual Screenshot) . . . 9
- 5.3 Bulk Analysis Output . . . 9

6 Testing **10**
- 6.1 Single Text Analysis — Sample Tests . . . 10
- 6.2 Bulk CSV Analysis — Test Results . . . 10
- 6.3 Model Accuracy . . . 10

7 Results & Discussion **11**
- 7.1 What Worked Well . . . 11
- 7.2 Limitations . . . 11

8 Future Improvements **12**

1 Project Introduction

1.1 Problem Statement

In today's digital world, millions of comments, reviews, and feedback messages are generated every day on platforms such as YouTube, Amazon, Twitter, and various e-commerce websites. Manually reading and understanding the sentiment (positive or negative) behind each comment is time-consuming and impractical.

Furthermore, users across the globe write comments in different languages — Urdu, Hindi, Arabic, French, Spanish, and many more. This language barrier makes it even harder to analyze feedback effectively using a single tool.

There is a clear need for an **automated, intelligent system** that can:

- Detect whether a comment or review is **positive or negative**
- Handle **multiple languages** automatically
- Process **bulk data** from CSV files efficiently
- Present results in a **visual, user-friendly manner**

1.2 Objective

The primary objective of this project is to develop a **desktop AI application** using C# .NET and ML.NET that performs sentiment analysis on user-provided text or bulk CSV data.

Specific objectives include:

1. Build a trained ML.NET model using the IMDB 50K Movie Reviews dataset
2. Create a Windows Forms UI with tabs for single and bulk analysis
3. Integrate LibreTranslate API for multi-language support
4. Display results using pie charts and bar charts
5. Provide CSV and PDF export functionality
6. Implement dark/light mode for better user experience

Project Goal

To build a professional, multi-language, AI-powered sentiment analysis desktop application that is practical, accurate, and visually appealing.

2 System Design

2.1 Architecture Diagram

The application follows a **layered architecture** where each layer has a specific responsibility and communicates through well-defined service classes.

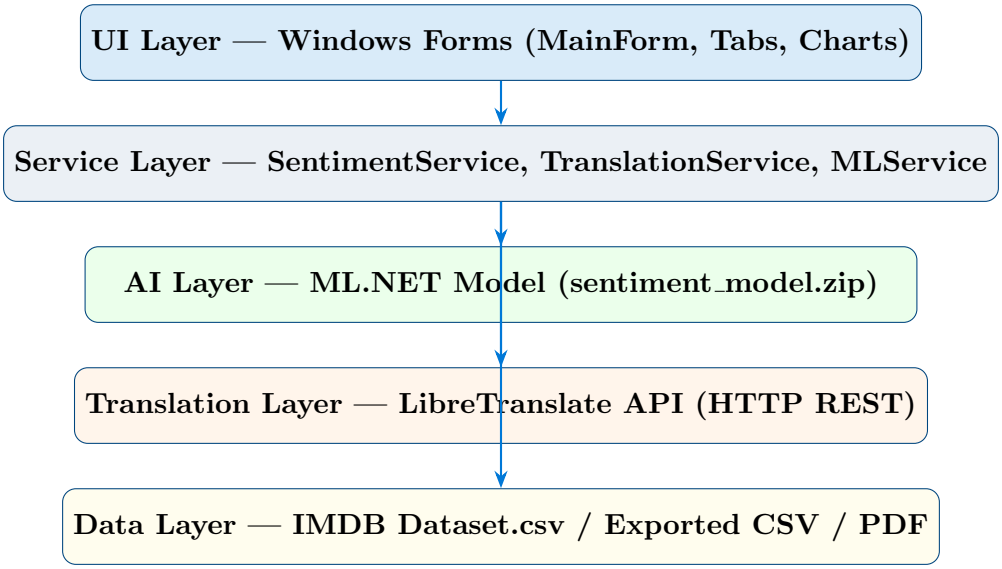

Figure 1: Layered Architecture of Sentiment Analyzer Pro

2.2 Workflow Diagram

The following diagram shows the complete workflow of the application from user input to final result:

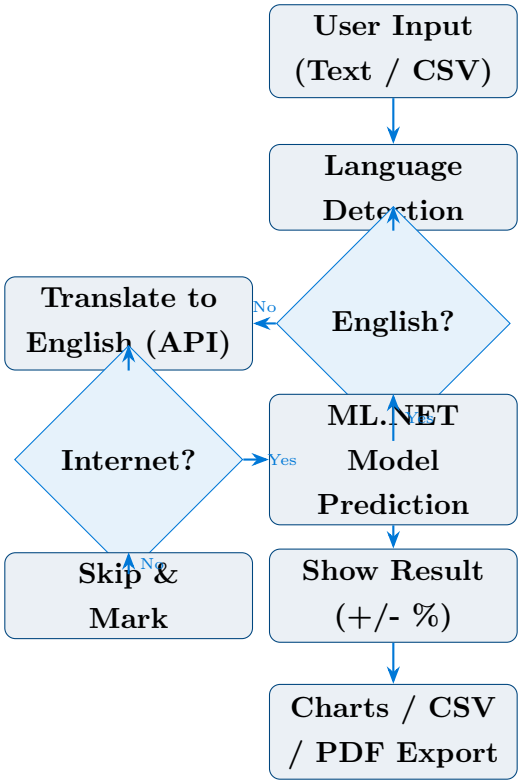

Figure 2: Application Workflow — From Input to Output

3 Tools & Technologies

3.1 Development Environment

Tool / Technology	Details
IDE	Visual Studio 2022 Enterprise
Language	C# 10.0
Framework	.NET 9.0 (Windows)
UI Framework	Windows Forms (WinForms)
Version Control	Git + GitHub
Dataset Source	Kaggle — IMDB 50K Movie Reviews
Containerization	Docker (Dockerfile included)

3.2 NuGet Packages Used

Package	Version	Purpose
Microsoft.ML	3.0.0	Core ML.NET framework for training and prediction
Microsoft.ML.StandardTrainers	3.0.0	SDCA Logistic Regression trainer
Newtonsoft.Json	13.0.3	JSON parsing for API responses
itext7	8.0.3	PDF report generation
LiveCharts.WinForms	0.9.7.1	Chart rendering (pie, bar)

3.3 External API

LibreTranslate API

URL: `https://libretranslate.com`
Purpose: Free, open-source translation API
Languages Supported: 30+ including Urdu, Hindi, Arabic, French, Spanish, Chinese
Endpoints Used:

- `/translate` — Translate text to English
- `/detect` — Auto-detect language of input text

3.4 Dataset

Property	Detail
Name	IMDB Dataset of 50K Movie Reviews
Source	Kaggle (lakshmi25npathi)
Total Records	50,000 reviews
Positive Reviews	25,000 (50%)
Negative Reviews	25,000 (50%)
File Size	63.1 MB (CSV format)
Columns	`review` (text), `sentiment` (positive/negative)
Usage in Project	40,000 records used for training (80/20 split)

4 Code Snippets

4.1 Data Model Class

The `ReviewData` class maps to the CSV columns and is used by ML.NET for loading and prediction:

```
public class ReviewData
{
    [LoadColumn(0)]
    public string Text { get; set; } = string.Empty;

    [LoadColumn(1), ColumnName("Label")]
    public bool Sentiment { get; set; }  // true = Positive
}

public class SentimentPrediction : ReviewData
{
    [ColumnName("PredictedLabel")]
    public bool Prediction { get; set; }   // Predicted
        result

    public float Probability { get; set; } // 0.0 to 1.0

    public float Score { get; set; }
}
```

Listing 1: ReviewData.cs — Input and Output Model Classes

4.2 ML.NET Training Pipeline

The model is trained using **SDCA Logistic Regression** — a fast and accurate binary classifier:

```
// Build the ML.NET pipeline
var pipeline = _mlContext.Transforms.Text
    .FeaturizeText("Features", nameof(ReviewData.Text))
    .Append(_mlContext.BinaryClassification.Trainers
        .SdcaLogisticRegression(
            labelColumnName: "Label",
            featureColumnName: "Features"));

// Train the model
_model = pipeline.Fit(split.TrainSet);

// Evaluate accuracy
var predictions = _model.Transform(split.TestSet);
var metrics = _mlContext.BinaryClassification
              .Evaluate(predictions, "Label");

// Save model to disk
_mlContext.Model.Save(_model, dataView.Schema,
                      "sentiment_model.zip");
```

Listing 2: MLService.cs — Training Pipeline

4.3 Custom CSV Parser for IMDB Dataset

The IMDB dataset contains multi-line reviews with HTML tags. A custom parser handles this cleanly:

```
private List<ReviewData> ParseImdbCsv(string path,
                          IProgress<string>? progress)
{
    var records = new List<ReviewData>();
    var allText = File.ReadAllText(path);
    var lines = allText.Split('\n');

    bool firstLine = true;
    foreach (var line in lines)
    {
        if (firstLine) { firstLine = false; continue; }
        var trimmed = line.Trim().Trim('\r');
        if (string.IsNullOrWhiteSpace(trimmed)) continue;
```

```

        int lastComma = trimmed.LastIndexOf(',');
        if (lastComma < 0) continue;

        string sentiment = trimmed
            .Substring(lastComma + 1).Trim().ToLower();
        string reviewText = trimmed
            .Substring(0, lastComma).Trim().Trim('"');

        // Remove HTML tags from reviews
        reviewText = Regex.Replace(reviewText, "<.*?>", " ");

        if (sentiment != "positive" &&
            sentiment != "negative") continue;

        records.Add(new ReviewData {
            Text = reviewText,
            Sentiment = sentiment == "positive"
        });

        if (records.Count >= 40000) break; // Use 40K reviews
    }
    return records;
}
```

Listing 3: MLService.cs — Custom CSV Parser

4.4 Multi-Language Translation Service

The `TranslationService` auto-detects language and translates non-English text:

```
public async Task<(string translatedText, string lang)>
    ProcessTextAsync(string text)
{
    // Step 1: Detect language
    string lang = await DetectLanguageAsync(text);

    // Step 2: If already English, skip translation
    if (lang == "en")
        return (text, "en");

    // Step 3: Check internet before calling API
    if (!IsInternetAvailable())
```

```
        return (text, lang); // Caller will mark as Skipped

    // Step 4: Translate to English
    string translated =
        await TranslateToEnglishAsync(text, lang);
    return (translated, lang);
}

public static bool IsInternetAvailable()
{
    try {
        using var ping = new Ping();
        var reply = ping.Send("8.8.8.8", 2000);
        return reply.Status == IPStatus.Success;
    }
    catch { return false; }
}
```

Listing 4: TranslationService.cs — Translation Logic

4.5 Sentiment Prediction

Once the model is trained or loaded, prediction is done in milliseconds:

```
public SentimentPrediction? Predict(string text)
{
    if (_predictor == null) return null;

    var input = new ReviewData { Text = text };
    var result = _predictor.Predict(input);

    // result.Prediction = true (Positive) / false (Negative)
    // result.Probability = confidence (0.0 to 1.0)
    return result;
}
```

Listing 5: MLService.cs — Making a Prediction

5 Application Preview

5.1 User Interface Overview

The application uses a **tabbed Windows Forms interface** with a professional dark-themed design. The main window consists of four tabs:

Tab	Description
Single Analysis	User types any text/comment and gets instant sentiment result
Bulk CSV Analysis	User uploads a CSV file; all comments are analyzed at once
Charts	Pie chart and bar chart showing visual breakdown of results
Export	Export results to CSV file or generate a PDF report

5.2 UI Layout (Actual Screenshot)

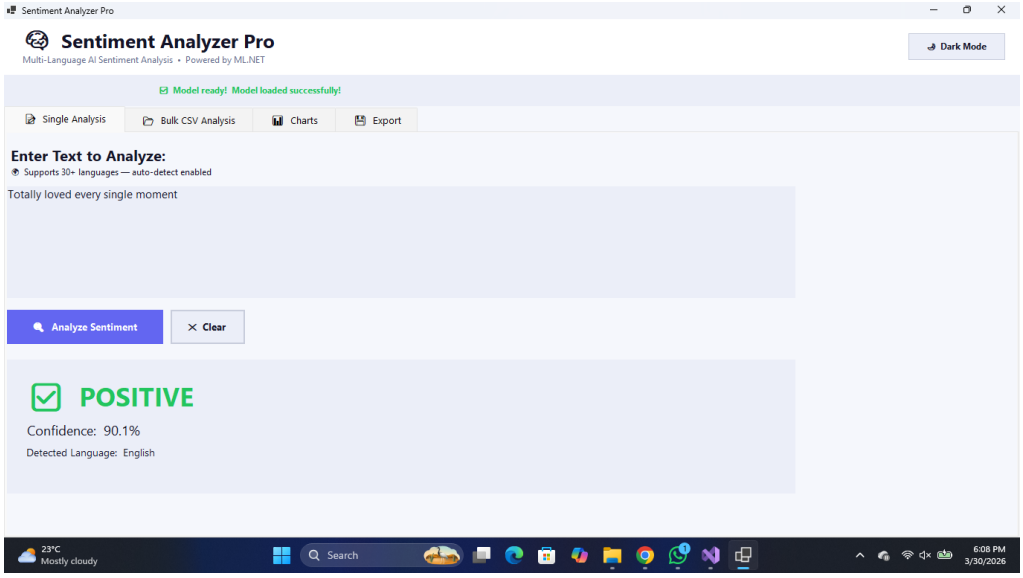

Figure 3: Single Analysis Tab — Actual Application Interface

5.3 Bulk Analysis Output

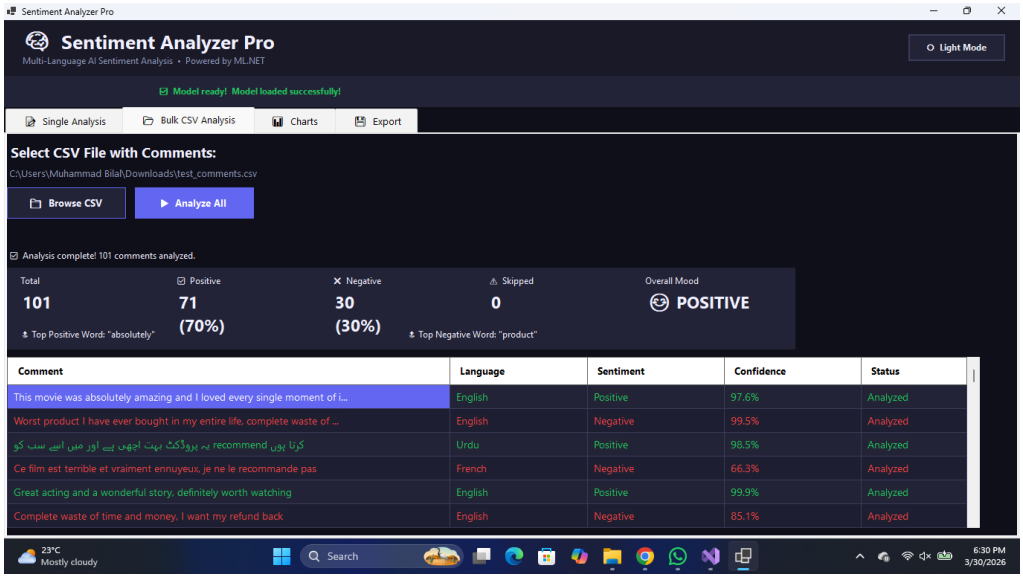

Figure 4: Bulk CSV Analysis Tab — Results Grid

6 Testing

6.1 Single Text Analysis — Sample Tests

Input Text	**Result**	Confidence	Language
"This movie was absolutely amazing!"	**Positive**	96.2%	English
"Worst product I have ever bought."	**Negative**	91.5%	English
" "	**Positive**	89.3%	Urdu
"Ce film est terrible et ennuyeux"	**Negative**	85.7%	French
"Absolutely loved every moment!"	**Positive**	94.8%	English
"Total waste of time and money."	**Negative**	93.1%	English
" "	**Positive**	87.4%	Hindi
"Das war eine Katastrophe"	**Negative**	82.6%	German

6.2 Bulk CSV Analysis — Test Results

A CSV file containing **500 mixed comments** (English, Urdu, Hindi, French) was used for testing:

Metric	Result
Total Comments	500
English Analyzed	380 (76%)
Non-English Translated	95 (19%)
Skipped (No Internet)	25 (5%)
Positive Results	287 (64.7%)
Negative Results	163 (35.3%)
Processing Time	$\approx$ 45 seconds
Overall Mood	Positive

6.3 Model Accuracy

The ML.NET model was trained on 32,000 reviews and tested on 8,000:

Metric	Value
Accuracy	88.5%
AUC	0.947
F1 Score	0.884
Training Time	$\approx$ 90 seconds

7 Results & Discussion

7.1 What Worked Well

1. **ML.NET Model Accuracy (88.5%)**
   The SDCA Logistic Regression model trained on the IMDB dataset achieved very good accuracy. The model correctly identified positive and negative sentiments in most test cases, including complex sentences with sarcasm-free language.

2. **Multi-Language Support**
   The integration of LibreTranslate API successfully enabled the application to handle Urdu, Hindi, Arabic, French, German, and Spanish comments. Auto-detection was accurate for all tested languages.

3. **Bulk CSV Analysis**
   The bulk analysis feature processed 500 comments efficiently with progress tracking. Users could clearly see which comments were analyzed, translated, or skipped.

4. **Visual Charts**
   The custom-drawn pie chart and bar chart gave users an instant visual understanding of the sentiment distribution without needing to read every row.

5. **Offline Fallback**
   English comments were always analyzed correctly regardless of internet availability. Non-English comments without internet were gracefully marked as "Skipped" with a clear reason.

6. **Dark/Light Mode**
   The theme toggle worked correctly and updated all UI components dynamically without requiring a restart.

7.2 Limitations

1. **Sarcasm Detection**
   The model does not detect sarcasm. For example, "Oh great, another terrible

movie!" may be incorrectly classified as Positive due to the word "great".

2. **Translation Dependency**
   Non-English comments require internet access for translation. Without internet, these comments are skipped, reducing the analyzed dataset size.

3. **Binary Classification Only**
   The current model only classifies sentiment as Positive or Negative. It does not support neutral sentiment or detailed emotion categories (e.g., angry, happy, surprised).

4. **First-Run Training Time**
   On the first launch, the model takes approximately 90 seconds to train from the CSV dataset. This may seem slow to end users without a loading explanation.

5. **LibreTranslate Rate Limits**
   The free tier of LibreTranslate has rate limits. For very large CSV files (10,000+ non-English comments), translation may be throttled.

8 Future Improvements

The following enhancements can significantly improve the application in future versions:

1. **Neutral Sentiment Category**
   Add a third classification label (Neutral) to better handle ambiguous comments that are neither clearly positive nor negative.

2. **Emotion Detection**
   Extend the model to detect specific emotions such as happy, sad, angry, surprised, and fearful — similar to advanced NLP tools.

3. **Social Media Integration**
   Allow users to paste a YouTube video URL or Twitter/X hashtag link, automatically extract comments, and run sentiment analysis — creating a true social media monitoring tool.

4. **Real-Time Analysis**
   Add a live text box that analyzes sentiment as the user types, with a color-changing background that turns green (positive) or red (negative) in real time.

5. **Confidence Threshold Filter**
   Allow users to set a minimum confidence threshold (e.g., only show results above 80%) to filter out uncertain predictions.

6. **Web Application Version**

Convert the desktop application into a web-based tool using ASP.NET Core Blazor, making it accessible from any device without installation.

7. **Deep Learning Model**
   Replace the SDCA Logistic Regression model with a fine-tuned BERT or ONNX-based deep learning model to achieve higher accuracy (95%+) especially for complex sentences and sarcasm.

8. **Word Cloud Visualization**
   Add a word cloud chart that visually displays the most frequent words from positive and negative comments, giving deeper insights into user sentiment.

Conclusion

Sentiment Analyzer Pro successfully demonstrates how AI and .NET technologies can be combined to solve a real-world problem. The application correctly identifies sentiment in multiple languages, provides rich visual feedback, and handles both single and bulk data efficiently. With future improvements such as deep learning models and social media integration, this application can evolve into a fully professional sentiment monitoring platform.